How Atlassian Reduced Latency by 70% with Sidecars

Source: Arpit Bhayani Dissection • Atlassian Infrastructure
Focus: Tenant Contact Service (TCS) & Polyglot Client Bottlenecks

1. The Problem: Atlassian's Tenant Contact Service (TCS)

Atlassian powers global collaboration across Jira, Confluence, and Bitbucket. The **Tenant Contact Service (TCS)** is a Tier-0 infrastructure microservice responsible for resolving organization metadata, billing entitlements, and tenant routing. Every user request across the ecosystem relies on TCS, imposing strict latency budgets in the low single-digit milliseconds.

Although TCS backend instances executed queries in sub-millisecond times, consumer services reported severe latency bloat ($100 ext{ ms} - 300 ext{ ms}$) and sporadic timeouts.

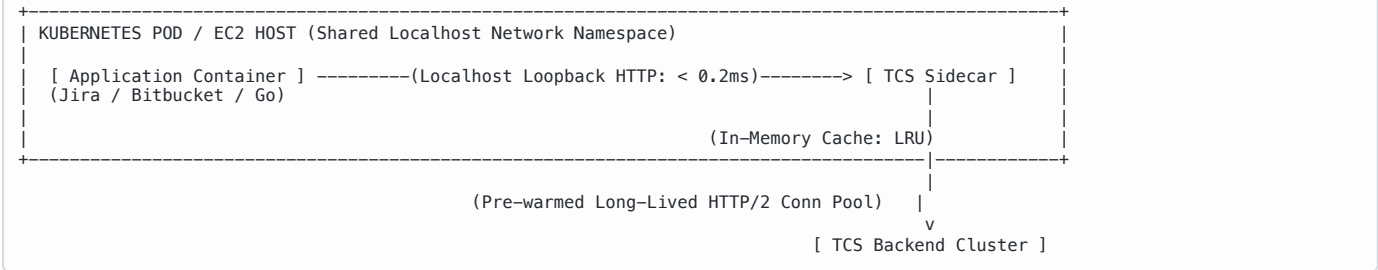
2. Root Cause: The Polyglot Client SDK Bottleneck

```
Consumer Service (e.g., Jira in Java, Bitbucket in Python):
1. Makes unpooled HTTP connection -> TCP 3-way handshake + TLS negotiation (30ms lost)
2. Issues synchronous GET /tenant/123/contact
3. Opens second fresh connection -> TCP + TLS overhead (30ms lost)
4. Issues synchronous GET /tenant/123/billing
5. Lacks local caching -> same static data re-queried across loop iterations
-------------------------------------------------------------------------------
Result: Over 90% of observed latency was wasted on client-side connection churn!
```

> **The Strategic Dilemma:** Atlassian could attempt to "Solve for One" (educate individual product teams and write shared client SDKs across Java, Python, Go, Node.js) or "Solve for All" (deploy an architectural pattern that abstracts network mechanics entirely). Shared libraries inevitably fail at scale due to dependency drift and maintenance burdens. Atlassian chose the **Sidecar Pattern**.

3. Sidecar Architectural Topology

A lightweight, language-agnostic proxy process runs in the **same pod / container host** as the application:



4. Key Architectural Optimizations Inside the Sidecar

By moving communication logic into a dedicated sidecar, the platform team implemented production-grade network optimizations transparently:

A. Pre-Warmed Long-Lived HTTP/2 Connection Pooling

The sidecar establishes and maintains permanent HTTP/2 connections with TCS backends. TCP and TLS handshakes are executed once at startup and amortized over millions of requests.

B. Transparent Multiplexing & Batching

Multiple sequential or concurrent calls from the application are automatically multiplexed over single connections or combined into bulk batch requests:

```
Application Thread A: GET /tenant/101  ---Application Thread B: GET /tenant/102   ---> [ Sidecar Proxy ] ===> [ Bulk Backend Query ]
Application Thread C: GET /tenant/103  ---/
```

C. High-Speed Local In-Memory Caching

Tenant metadata changes infrequently. The sidecar maintains an in-memory LRU cache with a short TTL (e.g. 60s). Over **85% of queries** are answered directly from local RAM in under $0.3 ext{ ms}$, completely eliminating network traversals.

5. Quantitative Performance Impact: 70% Latency Drop

Metric	Before (Polyglot Client SDKs)	After (TCS Sidecar Pattern)	Improvement
Client Observed p99 Latency	$120 ext{ ms} - 250 ext{ ms}$	**$35 ext{ ms}$ (Down to $0.3 ext{ ms}$ on cache hit)**	**70% to 90% reduction**
TCS Backend Request Volume	Millions of redundant sequential calls	Dramatically reduced via local cache	Backend CPU load dropped 65%
TCP / TLS Handshake Churn	Continuous socket open/close cycles	100% pre-warmed persistent connections	Zero connection setup latency
Upgrade Governance	Months of coordination across 4 languages	Single sidecar binary deployment	Instantaneous protocol updates

Trade-offs & First-Principles Systems Lessons

6. When to Use (and When to Avoid) Sidecars

The sidecar pattern offers immense architectural power, but introduces operational overhead that must be balanced:

Architectural Context	Recommended Approach	Core Justification
Polyglot Microservices (Java, Python, Go, Node)	**Sidecar Pattern**	Eliminates the cost of maintaining identical client networking libraries in 4+ languages.
Cross-Cutting Infrastructure (mTLS, Tracing, Retries)	**Sidecar Pattern**	Centralizes security certificates, OpenTelemetry, and circuit breaking in one layer.
Homogeneous Monoliths (100% Go or 100% Java)	**In-Process Library**	Shared library in a single language avoids extra container memory and IPC hop overhead.
Serverless / Edge Functions (AWS Lambda)	**In-Process / Managed Gateway**	Sidecar processes inflate cold start durations and memory footprints in ephemeral runtimes.

7. First-Principles System Design Takeaways

- **Systems Architecture Trumps Human Discipline:** Counting on dozens of independent engineering teams to correctly implement connection pooling, retries, and jitter fails at scale. Abstracting networking into an out-of-process sidecar guarantees universal reliability.
- **Localhost Loopback is Cheap:** Modern Linux Unix domain sockets and loopback interfaces transfer data in single-digit microseconds ($< 0.1 ext{ ms}$). This micro-overhead is negligible compared to the tens of milliseconds saved by pre-warmed connection pools and local in-memory caching.
- **Decouple Application Code from Infrastructure Evolution:** Upgrading protocols (e.g., migrating from HTTP/1.1 to gRPC or rolling out mTLS rotation) occurs entirely inside the sidecar container without recompiling or touching application repositories.